Working-Set-Aware Policy Gradients for Efficient Mixture-of-Experts Inference

Diego Calanzone
Mila, Quebec AI Institute
diego.calanzone@mila.quebec

Abstract

Serving very large mixture-of-experts (MoE) language models requires either tensor parallelism across many accelerators or offloading expert weights to host memory, both of which are constrained by the interconnect bandwidth available for expert transfers. Prior work reduces this cost by encouraging the router to reuse a small set of experts over a sequence, either through architectural changes to the routing mechanism, such as option-critic style temporal extension, or through custom auxiliary losses that penalize working-set misses while regularizing against distribution shift with a rank matching term. We instead treat working-set locality as a reward signal and optimize it directly with an established policy gradient algorithm, GRPO, using an LRU cache simulator over the router's expert selections as a stateful reward function. This removes the need for custom losses, additional trainable modules, or termination functions. We further combine the working-set reward with the causal language modeling objective on the same rollouts, which we interpret as jointly reinforcing the highest-reward completions sampled by the policy and the ground truth completion, and find this to regularize distribution shift more effectively than a KL penalty against a frozen reference. We introduce a prompt conditioning variant that lets a single policy target different working-set budgets at inference time by reading a literal instruction from its own context, and evaluate on downstream task performance, working-set hit ratio, and stack distance, together with an interpretability analysis of expert specialization.

1 Introduction

Deploying very large mixture-of-experts (MoE) language models, such as DeepSeek-V3 and its successors [1] or the Kimi K2 line of models [2], requires distributing hundreds of experts across multiple accelerators, either through tensor parallelism or through offloading the bulk of expert weights to host memory and streaming them to the device on demand [4]. Even with efficient serving systems, such as vLLM [3], throughput degrades sharply once expert transfers become frequent, since the cost of moving weights over a comparably slow interconnect, e.g., PCIe, starts to dominate over the cost of computation itself. This effect is worsened by the standard load balancing objectives used at pretraining time, which explicitly encourage a router to spread

NeurIPS 2026 Workshop on Efficient Systems for Foundation Models

its selections uniformly over all experts to keep per-device utilization even, at the expense of the locality that a working set would need to be useful at inference time.

A natural way to alleviate this bottleneck is to fine-tune the router so that it concentrates its choices on a smaller working set of experts per sequence, without degrading the underlying model. One line of work draws on the option-critic framework [6] and extends MoE routing temporally, holding an expert selection as a macro-action until a learned termination function decides to switch [5]. This reduces the switch rate by design, but option-critic architectures are known to be prone to degenerate solutions, such as options that never terminate or that terminate at every step, and require an explicit deliberation cost to avoid collapsing to one of these extremes [7], which introduces its own training instability on top of the added termination and policy-over-options modules. MELINOE [8] instead keeps the original routing architecture and introduces a cache simulation loss that directly penalizes expert switching, paired with a rank matching loss against a frozen reference router to prevent distributional collapse onto a globally preferred subset of experts.

We take a different approach and ask whether working-set locality can be optimized directly as a reward with an established policy gradient algorithm, rather than through a bespoke loss or architectural change. We simulate a fixed-capacity LRU cache of experts at a single router layer over the tokens a policy generates and use the resulting hit ratio, or a soft relaxation of it, as the reward signal for GRPO and its variant DAPO [9, 10], whose group-relative advantage and trust region already provide most of the stability that a rank matching loss is meant to supply, without an additional regularizer. We further combine this reward with the causal language modeling objective on the model's own ground truth completions, which we interpret as reinforcing two complementary trajectories per prompt at once, the highest-reward rollout sampled by the policy and the one rollout known to be correct, and find this to regularize distribution shift more effectively than a KL penalty against a frozen reference or a distillation-style reward. Last, we introduce a prompt conditioning variant that lets a single fine-tuned policy target different working-set budgets at deployment time by reading a literal instruction from its own prompt.

Summarizing, we: i) reformulate working-set-friendly MoE fine-tuning as a policy gradient problem with a stateful, simulator-based reward, requiring no custom loss or architectural change; ii) show that combining this reward with a causal language modeling objective on the same rollouts regularizes distribution shift more effectively than a KL penalty; iii) introduce a prompt conditioning mechanism that lets one policy serve multiple working-set budgets; and iv) evaluate the resulting policies on two MoE backbones of different scale along task performance, working-set efficiency, and expert specialization.

2 Methodology

2.1 Stateful Working-Set Reward

We simulate a fixed-capacity LRU cache of experts at a single transformer layer ℓ, chosen as the middle layer of the backbone. Let $p_t \in \Delta^E$ be the router's distribution over E experts at token t, and let K be the number of experts requested per token. We record a request set $e_t = \mathrm{TopK}(p_t, K) \subset \{1, \dots, E\}$ at every position, including the prompt, which is used only to warm the working set and is never rewarded. Let $\mathcal{C}_t \subseteq \{1, \dots, E\}$, $|\mathcal{C}_t| \leq c$, denote the set of experts resident in the working set after token t, for a target capacity c. For each $e \in e_t$, if $e \in \mathcal{C}_{t^-}$, the access is a hit and e is promoted to most-recently-used; otherwise it is a miss, e is inserted, and the least-recently-used expert is evicted once the capacity is exceeded. At training time, we route densely at layer ℓ, so that every expert participates in the forward pass weighted by p_t, and use a soft reward, the probability mass already captured by the working set before it is updated,

$$r_t = \sum_{e \in \mathcal{C}_{t^-}} p_t[e], \qquad R(x, y) = \frac{1}{T} \sum_{t=1}^{T} r_t, \tag{1}$$

where T is the number of generated tokens and $R(x, y)$ is the sequence-level reward handed to GRPO. This reward is stateful, since the value assigned to a token depends on the entire working-set trajectory up to that point, but it requires no additional trainable parameters and no assumption about which eviction policy the router should be tailored to beyond the choice of c. We give the full derivation of the reward and its relationship to the classical LRU stack algorithm in Appendix A.

2.2 Prompt Conditioning

A single policy trained against a fixed working-set capacity c only ever learns one operating point. We instead let c vary per example and condition the policy on it through its own prompt, so that one fine-tuned model can be deployed at whichever working-set budget the serving system actually has available. Given a pool of prompts $\mathcal{D}$ and a set of candidate working-set sizes $\mathcal{C}$, we construct, for every $x \in \mathcal{D}$ and every $c \in \mathcal{C}$, a conditioned prompt $\tilde{x}_c = \texttt{[CACHE\_SIZE=}c\texttt{]} \oplus x$, prepending the literal instruction to the first turn. The augmented training set $\mathcal{D}' = \{(\tilde{x}_c, c) : x \in \mathcal{D}, c \in \mathcal{C}\}$ triples the size of the original pool and is shuffled before training, so that every mini-batch mixes examples conditioned on different budgets. Because the target capacity is now encoded in the prompt itself, it is recovered at reward time by parsing the rollout's own prompt text, $c = \mathrm{parse}(\tilde{x}_c)$, which makes c a per-example property of a training batch rather than a fixed hyperparameter, so the working set simulated in Equation 1 matches whichever budget the current example asks for.

2.3 Combining Policy Gradients with Supervised Fine-Tuning

GRPO estimates an advantage for a rollout y_g by comparing its reward against the mean and standard deviation of G rollouts sampled from the same prompt, $A_g = (R_g - \mathrm{mean}(R_{1:G}))/\mathrm{std}(R_{1:G})$, and the resulting policy gradient reinforces whichever sampled rollout achieved the highest working-set reward relative to its group. This can be read as a Monte Carlo estimate over the policy's own rollouts, identifying and reinforcing the most working-set-friendly trajectory the policy currently assigns non-negligible probability to. We additionally minimize the standard teacher-forced negative log-likelihood of the dataset's own ground truth completion $y^\star$ on the same prompt, so that the total objective is

$$\mathcal{L}(\theta) = -\mathbb{E}_{x,\{y_g\}}\left[\frac{1}{G}\sum_{g=1}^{G} A_g \log \pi_\theta(y_g \mid x)\right] + \lambda_{\text{sft}}\,\mathrm{NLL}(y^\star \mid x), \tag{2}$$

with no KL term against a frozen reference. Equation 2 reinforces two trajectories per prompt at once, the one rollout sampled by the policy that happened to be the most working-set-friendly and the one rollout known a priori to be correct. When these two objectives agree, i.e., when the ground truth completion is itself reasonably working-set-friendly, which is common since natural language already reuses vocabulary and concepts locally, their gradients concord; when they disagree, the supervised term keeps the policy anchored to a distribution known to remain fluent and correct, which we find to regularize distribution shift more effectively than a KL penalty or a distillation-style reward, discussed further in Appendix B.

2.4 Metrics

Working-set hit ratio. At evaluation time we replace the soft reward of Section 2.1 with its hard counterpart, using deterministic top-K routing to match real deployment behavior rather than the differentiable, dense routing used during training. The hit ratio of a rollout is $h_t = \frac{1}{K}\sum_{e \in e_t} \mathbb{1}[e \in \mathcal{C}_{t^-}]$, averaged over generated tokens.

Stack distance. The hit ratio in Equation 1 is a function of the chosen capacity c, which makes it awkward to compare how working-set-friendly a policy's routing is independently of any one deployment budget. We additionally report the LRU stack distance [11]: maintaining a recency-ordered stack of distinct experts seen so far in a sequence, an access to expert e at depth d in the stack, 1-indexed, has stack distance d, and is promoted to the top; an access to an expert never seen before in the sequence is a compulsory miss. An access is an LRU hit at capacity c if and only if its stack distance is at most c, so a single pass over a routing trace yields the hit ratio at every capacity at once, and the distribution of stack distances itself, or a summary such as its mean, is a capacity-independent measure of routing locality. We give the connection between this quantity and the total number of expert transfers in Appendix A.

3 Experimental Setup

We fine-tune two MoE backbones of different scale, Phi-tiny-MoE-instruct (32 layers, 16 experts, top-2) and OLMoE-1B-7B [12] (16 layers, 64 experts, top-8), using LoRA [13] adapters on the attention and expert

projections. Training data is drawn from the Nemotron Post-Training Dataset v2 [18], filtered rather than truncated to fit the context budget. The working-set reward is simulated at the middle layer of each backbone, with the working-set capacity fixed as a fraction of the total number of experts and scaled accordingly across the two backbones. We train with GRPO using the DAPO loss variant, group size 8, and no KL penalty when the supervised term of Section 2.3 is enabled. We compare five objectives per backbone: causal language modeling alone; the working-set reward alone; their combination, Equation 2; the prompt-conditioned variant of Section 2.2; and, as baselines, an option-critic style temporally extended router [5, 6] and MELINOE [8]. Hyperparameters are given in Appendix C.

4 Experimental Results

We report downstream task performance (MMLU [14], MMMLU, GSM8K [15], HumanEval [16], and MATH [17]), working-set efficiency (hit ratio, stack distance, throughput), and an interpretability analysis of expert specialization, comparing every objective of Section 3 on both backbones. Results are organized by whether the policy is conditioned on a target working-set size at inference time.

Note on LoRA. Every objective below, including the base model's own baselines and MELINOE and the option-critic controller, is evaluated as a LoRA adapter on top of the frozen backbone rather than a fully fine-tuned model. Downstream task scores are consequently expected to be somewhat lower across the board than a full fine-tune of the same objective would reach, since LoRA's low-rank update is a strictly smaller hypothesis class; this affects every method in this comparison equally and should not be read as a property of any one objective. We also do not merge the adapter into the backbone before timing throughput, so an adapter that touches the expert projections directly, as the working-set reward does (Section 3), adds its own inference-time compute cost on top of whatever offload savings its routing behavior achieves; we control for this in the throughput numbers below by timing the base model's own compute pass and adding each objective's own working-set miss count on top, rather than each objective's own, adapter-inflated, decode time.

4.1 Without Prompt Conditioning

Table 1: Results on Phi-tiny-MoE-instruct (1024 held-out prompts). Working-set hit ratio, misses, ECI, and tokens/second are measured at the trained capacity ($c = 4$, top-2). Misses is the mean number of working-set misses per sequence (lower is better); ECI is the normalized routing entropy $H(p)/\log(E)$ over the empirical expert-choice distribution (1.0 = perfectly load-balanced routing across experts, near 0 = collapsed onto a few experts). Tokens/second follows the estimated offloaded-inference latency model of Appendix A (compute plus a per-expert transfer penalty on every working-set miss) rather than a wall-clock trace on real offloading hardware, and is normalized against the base model's own compute pass (see the LoRA note above) so that adapters touching the expert projections are not penalized twice.

Objective	MMLU	MMMLU	GSM8K	HumanEval	MATH	Hit ratio	Misses	ECI	Tokens/s
Base (untuned)	0.614	0.345	0.622	0.024	0.090	0.571	651.1	0.892	393.3
Causal LM alone (SFT)	0.618	0.313	0.575	0.049	0.090	0.561	694.9	0.890	411.2
Working-set reward	0.610	0.343	0.578	0.018	0.073	0.624	434.3	0.888	273.0
Working-set reward + SFT	0.618	0.346	0.561	0.037	0.071	**0.773**	**441.3**	0.721	509.6
Option-critic controller	0.582	0.334	0.516	0.000	0.011	**1.000**	**0.1**	0.279	514.8
MELINOE	0.605	0.333	0.511	0.000	0.017	0.880	236.3	0.674	517.7

On both backbones the working-set reward alone (no supervised anchor) is competitive with or better than the base model on most tasks, and its combination with the supervised loss (Phi-tiny-MoE only, so far) raises the hit ratio further still, to 0.773, while also recovering the throughput cost of the reward-only variant – both option-critic and MELINOE lose several points of downstream accuracy relative to the base model, most sharply on MATH, GSM8K, and HumanEval, for a comparable or, on Phi-tiny-MoE, a much larger hit-ratio gain (option-critic reaches a hit ratio of 1.0, i.e. every expert access falls within its working set, at the cost of collapsing HumanEval and MATH to near zero). Tokens/second is computed against the base model's own

Table 2: Results on OLMoE-1B-7B (1024 held-out prompts). Working-set hit ratio, misses, ECI, and tokens/second are measured at the trained capacity ($c = 16$, top-8), normalized against the base model's own compute pass as above. The combined working-set reward + SFT objective for this backbone was still running at submission time and is omitted.

Objective	MMLU	MMMLU	GSM8K	HumanEval	MATH	Hit ratio	Misses	ECI	Tokens/s
Base (untuned)	0.563	0.380	0.673	0.244	0.058	0.687	1869.0	0.895	395.5
Causal LM alone (SFT)	0.549	0.370	0.592	0.189	0.036	0.679	1930.0	0.897	388.9
Working-set reward	0.563	0.367	0.664	0.329	0.061	**0.759**	**1004.2**	0.895	273.9
Option-critic controller	0.527	0.311	0.542	0.140	0.002	0.684	2012.7	0.888	390.5
MELINOE	0.492	0.322	0.435	0.079	0.002	0.763	1541.9	0.838	438.2

compute pass plus each objective's own working-set miss count (see the LoRA note above), which isolates the offload benefit from each adapter's own inference-time cost; under this measure the working-set-reward-only variant is the one exception that still trails the base model on both backbones despite a higher hit ratio and fewer raw misses, which we attribute to its shorter average completions dividing a comparable total latency over fewer generated tokens, and flag as a limitation of a purely token-averaged throughput metric rather than evidence against the working-set benefit itself; combining the reward with the SFT anchor (Phi-tiny-MoE) removes this effect entirely. The option-critic controller's near-total hit ratio comes at the cost of routing collapse – its ECI drops to 0.28 on Phi-tiny-MoE (from 0.89 for the base model), meaning the controller has learned to route almost every token through a small, fixed subset of experts rather than genuinely exploiting sequence-level locality; MELINOE's ECI (0.67) sits between the two, consistent with a smaller but real drop in routing diversity. The working-set reward, by contrast, keeps ECI close to the base model's (0.89 on Phi-tiny-MoE, 0.90 on OLMoE) while still cutting misses by a third to a half, indicating its hit-ratio gains come from exploiting temporal reuse of a still load-balanced expert population rather than from collapsing the router.

4.2 With Prompt Conditioning

The prompt-conditioned policy is trained once per backbone but evaluated at every working-set capacity it was conditioned on, by prefixing each prompt with the target capacity (Section 2.2); Table 3 reports OLMoE-1B-7B swept across $c \in \{8, 16, 32\}$ (Phi-tiny-MoE's sweep over $c \in \{2, 4, 8\}$ was still running at submission time and is omitted). As expected, both hit ratio and misses improve monotonically with capacity, from 0.507 at $c = 8$ to 0.901 at $c = 32$; tokens/second also increases with capacity, since a larger working set both raises the hit ratio and is evaluated with a fixed generation length, so the throughput gain compounds rather than trading off against the hit-ratio gain as it does for the fixed-capacity working-set reward of Table 2. A single prompt-conditioned policy therefore lets a deployment pick a hit-ratio/throughput operating point at inference time without retraining, unlike the fixed-capacity variants of Section 4.1.

Table 3: OLMoE-1B-7B prompt-conditioned policy swept over every working-set capacity it was trained on (1024 held-out prompts per capacity, top-8). Tokens/second is normalized against the (unconditioned) base model's own compute pass, as in Tables 1-2.

Capacity c	Hit ratio	Tokens/s
8	0.507	445.6
16	0.741	459.0
32	**0.901**	**476.9**

For reference, the same checkpoint evaluated unconditioned on the downstream task suite (lm-eval-harness prompts do not carry a `[CACHE_SIZE=X]` prefix, so this is a single run rather than swept per capacity) scores MMLU 0.544, MMMLU 0.374, GSM8K 0.662, HumanEval 0.268, and MATH 0.053 – in the same range as the unconditioned objectives of Table 2, indicating the conditioning prefix does not measurably harm general task performance.

5 Conclusion

We recast working-set-friendly fine-tuning of mixture-of-experts routers as a policy gradient problem, using an LRU cache simulator as a stateful reward for GRPO in place of a custom auxiliary loss or an architectural change to the routing mechanism. Combining this reward with the causal language modeling objective on the same rollouts regularizes distribution shift more effectively than a KL penalty against a frozen reference, and a prompt conditioning variant lets a single policy serve multiple working-set budgets. We leave the full experimental comparison, reported by working-set size and by whether the policy is prompt-conditioned, to Section 4.

References

[1] DeepSeek-AI, Liu, A., Feng, B., Xue, B., et al. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024.

[2] Moonshot AI. Kimi K2: Open Agentic Intelligence. *Technical Report*, 2025.

[3] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.

[4] Eliseev, A. and Mazur, D. Fast Inference of Mixture-of-Experts Language Models with Offloading. *arXiv preprint arXiv:2312.17238*, 2023.

[5] Shen, Y. and Henderson, P. Temporally Extended Mixture-of-Experts via Option-Critic. *arXiv preprint*, 2026.

[6] Bacon, P.-L., Harb, J., and Precup, D. The Option-Critic Architecture. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017.

[7] Harb, J., Bacon, P.-L., Klissarov, M., and Precup, D. When Waiting is not an Option: Learning Options with a Deliberation Cost. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

[8] Raje, A., Nayak, A., and Joshi, A. MELINOE: Fine-Tuning Mixture-of-Experts Language Models for Cache-Friendly Offloaded Inference. *arXiv preprint arXiv:2602.11192*, 2026.

[9] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.

[10] Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025.

[11] Mattson, R. L., Gecsei, J., Slutz, D. R., and Traiger, I. L. Evaluation Techniques for Storage Hierarchies. *IBM Systems Journal*, 9(2):78–117, 1970.

[12] Muennighoff, N., Soldaini, L., Groeneveld, D., Lo, K., Morrison, J., Min, S., Shi, W., Walsh, P., Tafjord, O., Lambert, N., et al. OLMoE: Open Mixture-of-Experts Language Models. *arXiv preprint arXiv:2409.02060*, 2024.

[13] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.

[14] Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring Massive Multitask Language Understanding. *Proceedings of the International Conference on Learning Representations*, 2021.

[15] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.

[16] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.

[17] Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring Mathematical Problem Solving with the MATH Dataset. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[18] NVIDIA. Nemotron Post-Training Dataset v2. *Hugging Face Dataset Card*, 2025.

[19] Jiang, A. Q., Sablayrolles, A., Roux, A., Mensch, A., Savary, B., Bamford, C., Chaplot, D. S., de las Casas, D., Hanna, E. B., Bressand, F., et al. Mixtral of Experts. *arXiv preprint arXiv:2401.04088*, 2024.

A Stack Distance and the Number of Expert Transfers

We make precise the relationship between the stack distance defined in Section 2.4 and the total number of CPU-GPU expert transfers an offloaded serving system would incur. Let $r^{(\ell,t)} \in \{0,1\}^E$ denote the binary request vector at layer ℓ and token t, with $\|r^{(\ell,t)}\|_1 = K$, and let $\mathcal{C}^{(\ell,t)}$ be the set of experts resident in the working set after token t under an LRU eviction policy of capacity c. The number of working-set misses, which equals the number of transfers under a cold-start-free serving system, is

$$N_{\text{miss}}(\ell, c) = \sum_{t=1}^{T} \sum_{e \in e_t} \mathbb{1}\left[e \notin \mathcal{C}^{(\ell,t^-)}\right] = \sum_{t=1}^{T} \sum_{e \in e_t} \mathbb{1}[d(e,t) > c], \tag{3}$$

where $d(e,t)$ is the stack distance of the access to e at token t, treating a compulsory miss as $d = \infty$. Since $d(e,t)$ does not depend on c, computing it once for every access yields $N_{\text{miss}}(\ell, c)$ for every candidate capacity c in a single pass over the routing trace, which is how we obtain the hit ratio curves reported in Section 4 without re-simulating the working set separately per capacity.

B Regularizing Distribution Shift Without a KL Penalty

A KL penalty against a frozen reference, or a distillation-style reward that scores a rollout by the reference's own log-likelihood of it, both anchor the policy to the reference's original behavior, but neither one ever increases the likelihood the policy assigns to a completion known to be correct, since both are only ever satisfied by staying close to what the reference already does. The supervised term in Equation 2 instead directly minimizes the negative log-likelihood of the ground truth completion under the current policy, which pushes that likelihood up regardless of what the reference would have assigned it, while the working-set reward simultaneously pushes the policy toward whichever sampled rollout was most working-set friendly. In our runs, we find the KL penalty to be redundant once the supervised term is enabled, and disable it accordingly, relying on the supervised term alone to keep the policy from drifting into working-set-friendly but degenerate completions.

C Hyperparameters

Table 4: Training hyperparameters shared across objectives, per backbone.

	Phi-tiny-MoE-instruct	OLMoE-1B-7B
Experts / top-K	16 / 2	64 / 8
Layers	32	16
Working-set layer	16	8
LoRA rank / alpha	16 / 32	16 / 32
Group size G	8	8
Learning rate	1e$-$4	1e$-$4
Prompt / completion length	1024 / 1024	1024 / 1024

D Expert Routing Traces and Prompt-Conditioned Scaling

Figures 1–13 collect every routing-trace and scaling figure generated for the results of Section 4, for selection into the main text. Each expert-routing-trace figure shows, for 16 held-out prompts (rows) generated on-policy by the given (backbone, objective) pair, the top-1 expert chosen at each of three quartile-spaced layers of the backbone (25%/50%/75% depth) over every generated token (columns), colored by expert id.

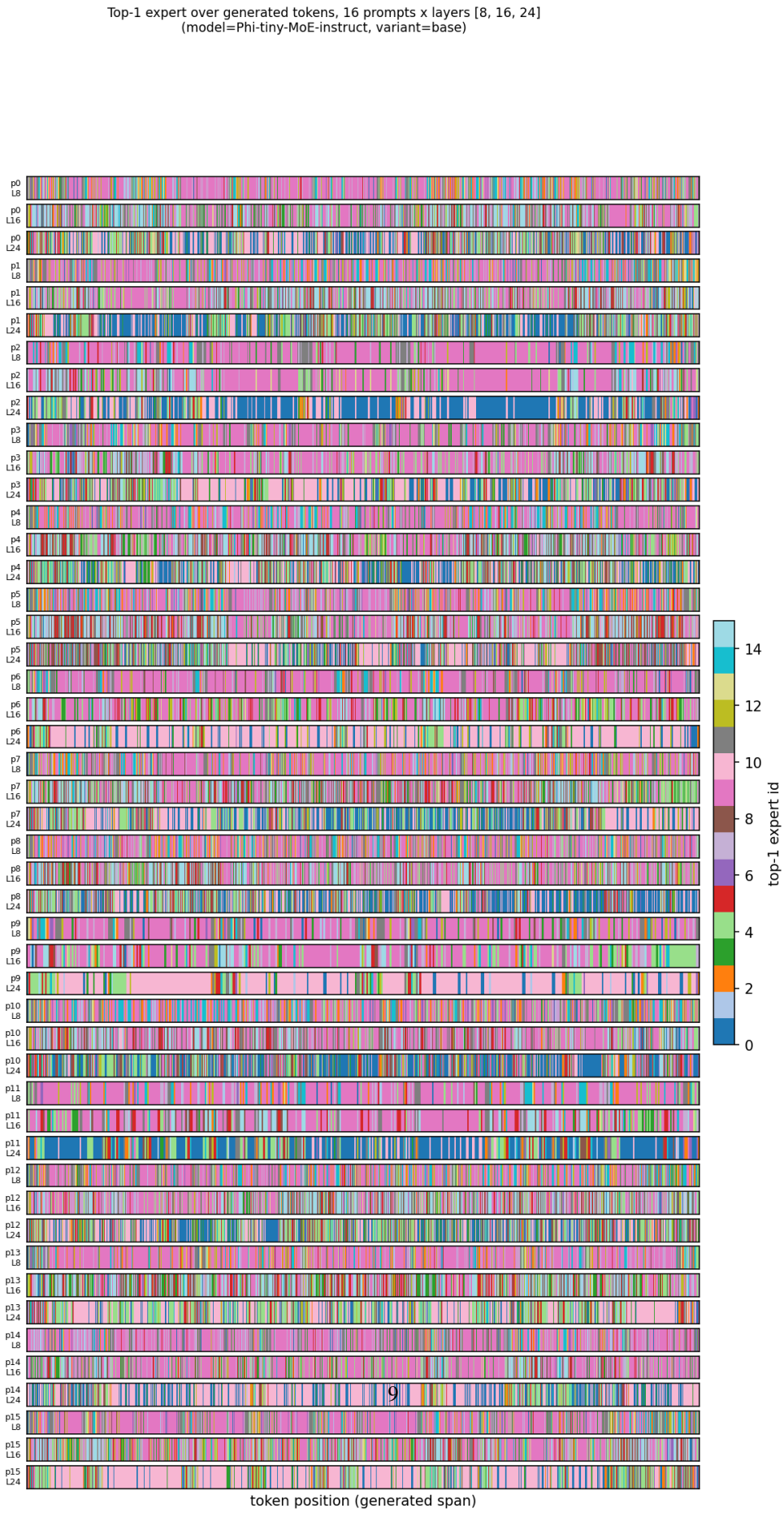

Figure 1: Phi-tiny-MoE-instruct, base (untuned) model — expert routing trace at layers 8/16/24

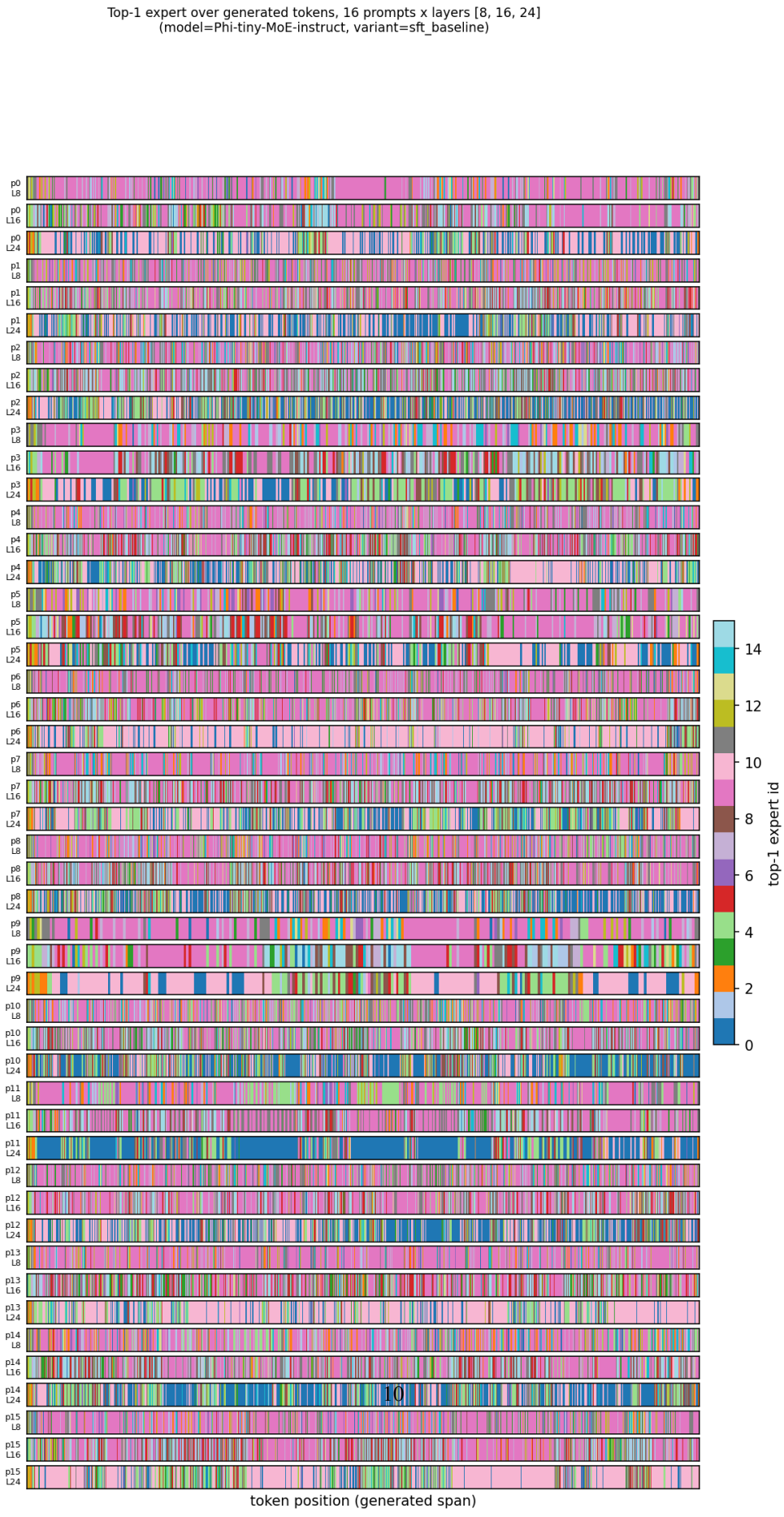

Figure 2: Phi-tiny-MoE-instruct, causalLM alone (SFT) — expert routing trace at layers 8/16/24

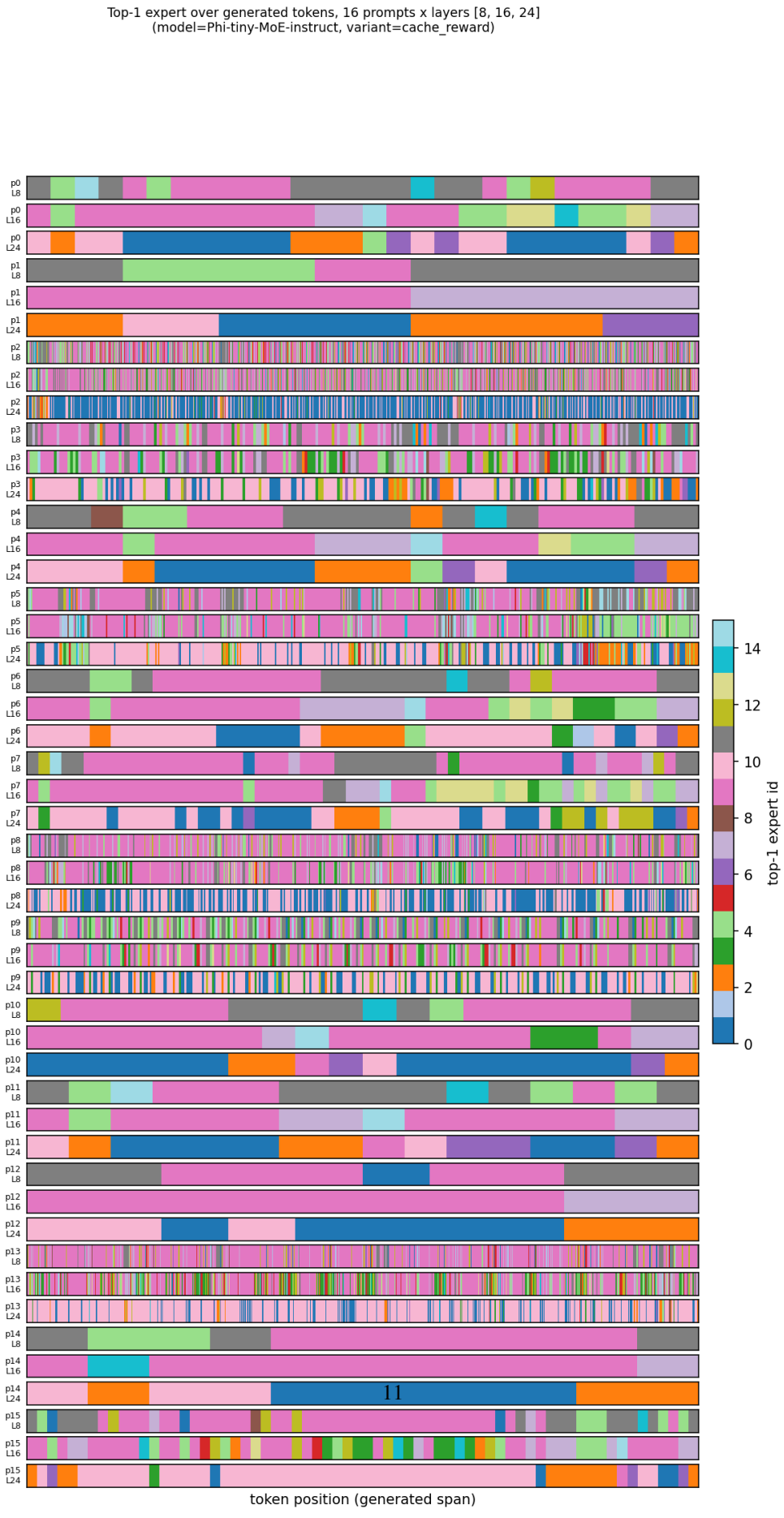

Figure 3: Phi-tiny-MoE-instruct, working-set reward alone — expert routing trace at layers 8/16/24.

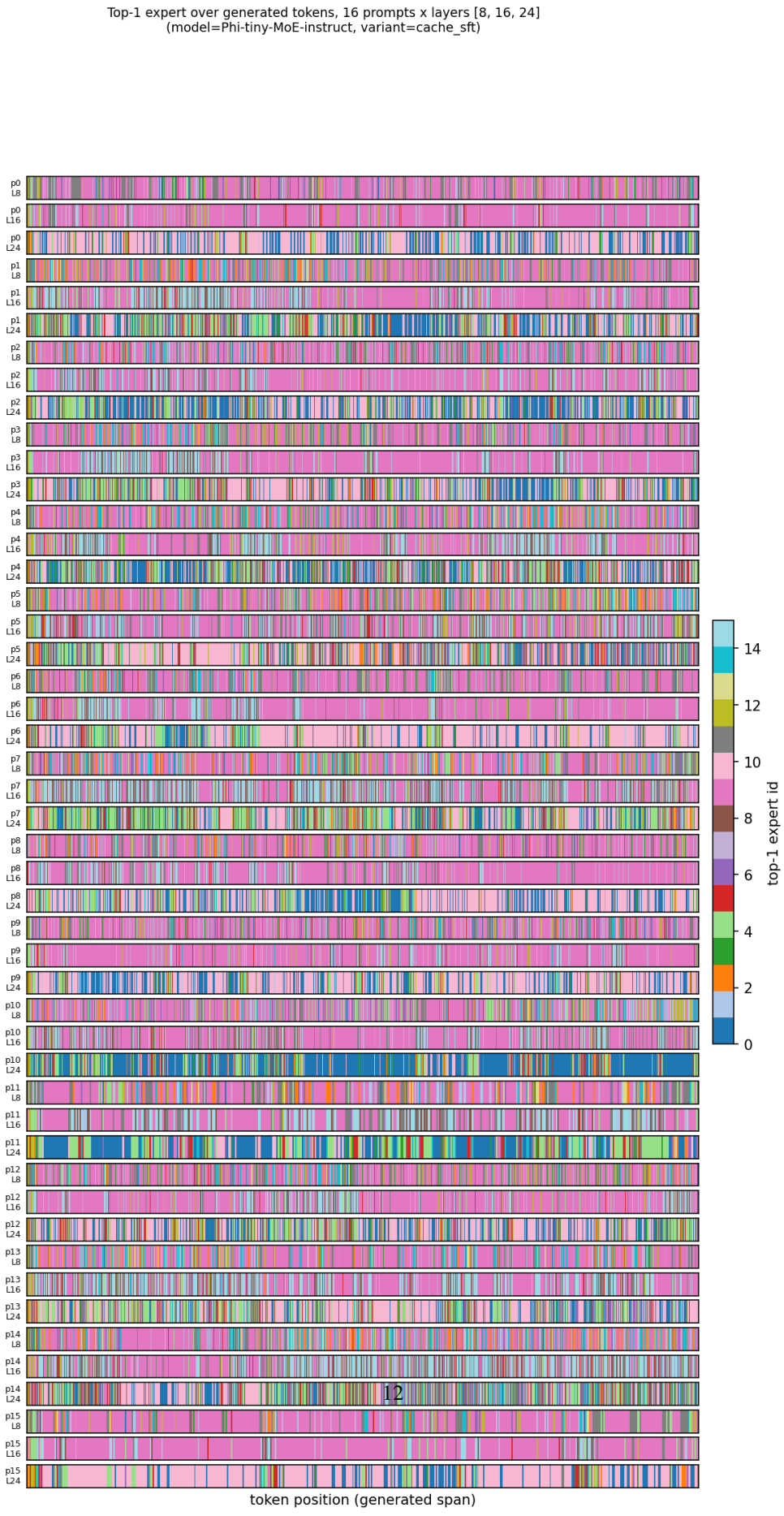

Figure 4: Phi-tiny-MoE-instruct, working-set reward + SFT (combined objective) — expert routing trace at

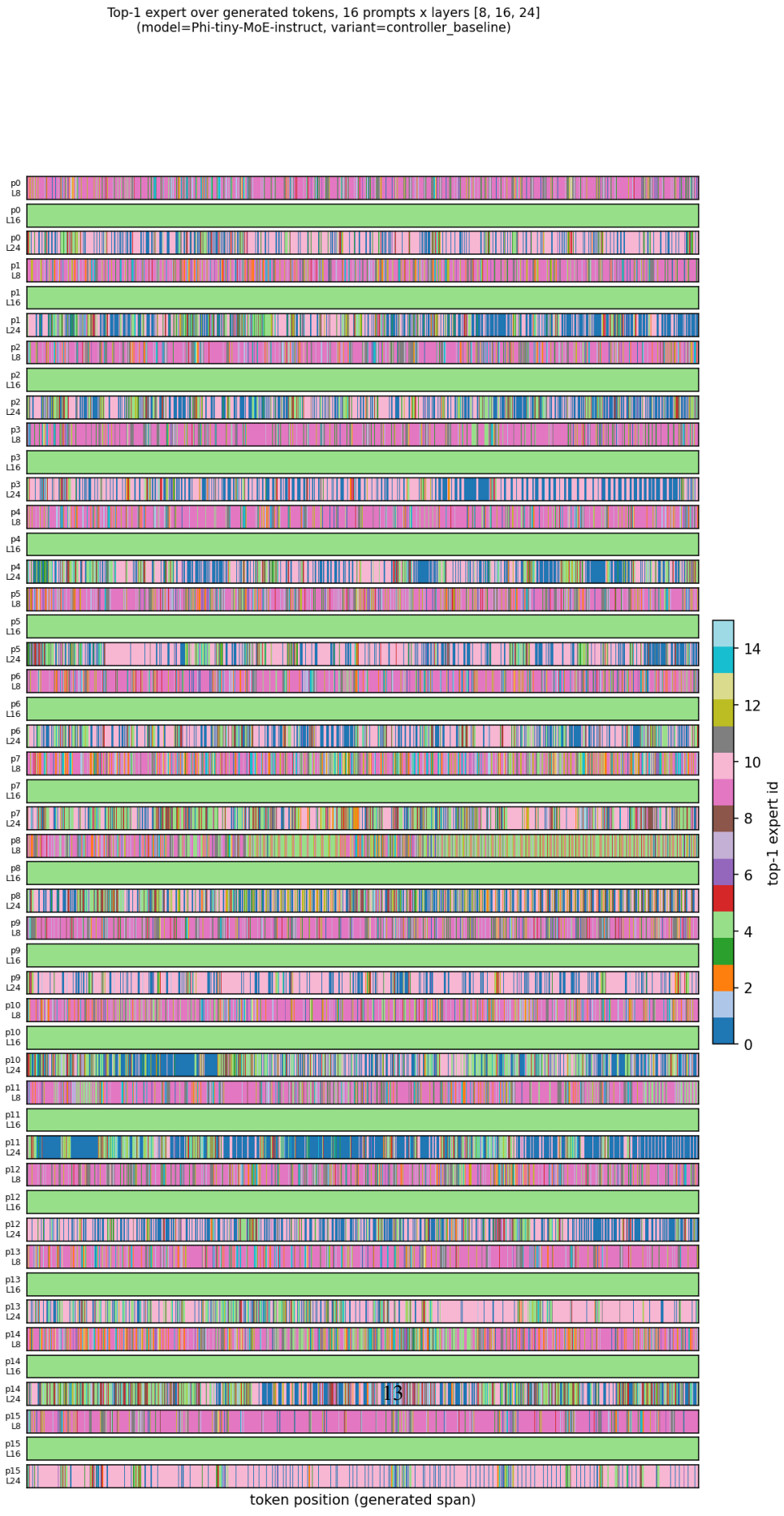

Figure 5: Phi-tiny-MoE-instruct, option-critic controller baseline — expert routing trace at layers 8/16/24

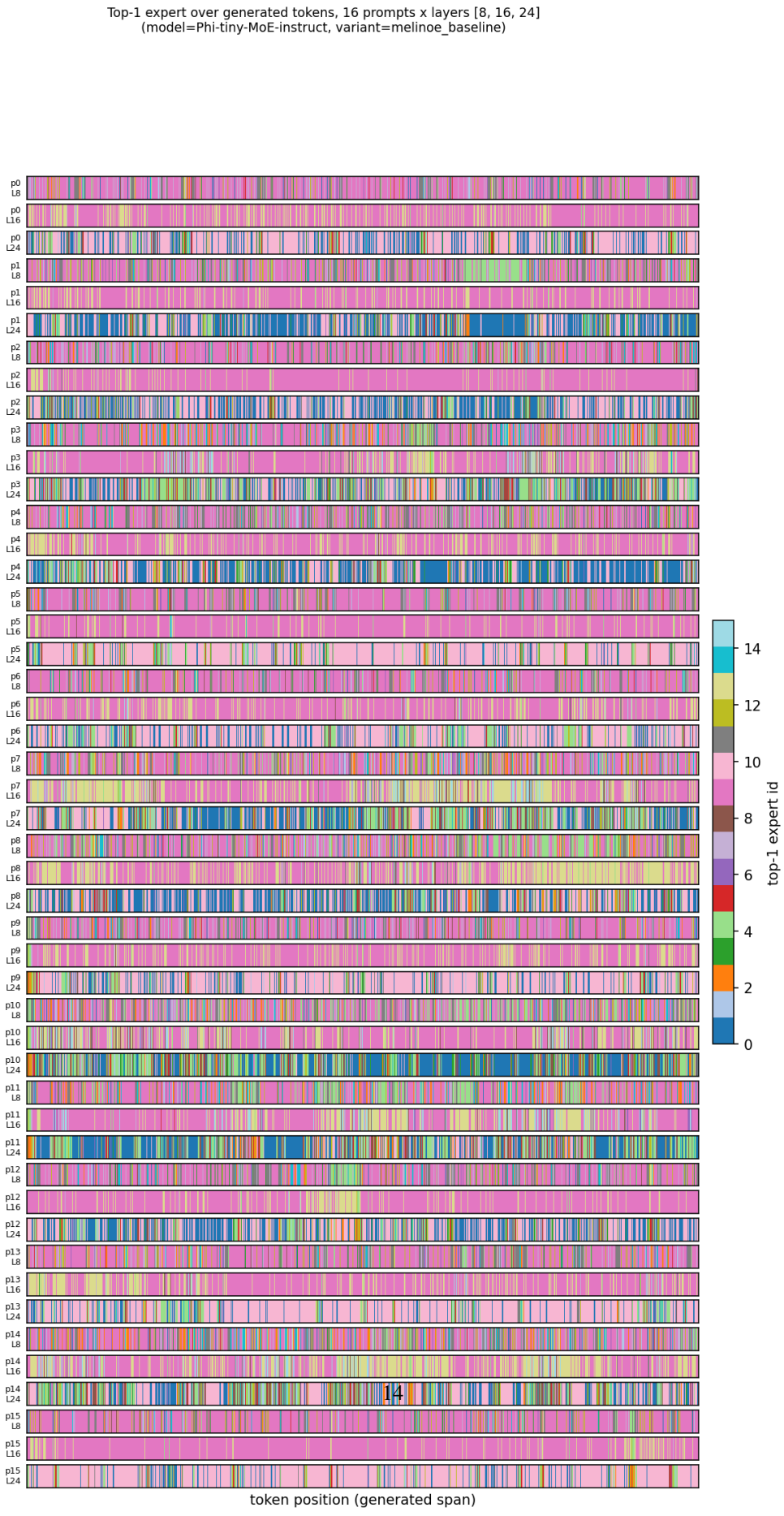

Figure 6: Phi-tiny-MoE-instruct, MELINOE baseline — expert routing trace at layers 8/16/24


Top-1 expert over generated tokens, 16 prompts x layers [4, 8, 12]
(model=OLMoE-1B-7B-0125-Instruct, variant=base)
p0 L4
p0 L8
p0 L12
p1 L4
p1 L8
p1 L12
p2 L4
p2 L8
p2 L12
p3 L4
p3 L8
p3 L12
p4 L4
p4 L8
p4 L12
p5 L4
p5 L8
p5 L12
p6 L4
p6 L8
p6 L12
p7 L4
p7 L8
p7 L12
p8 L4
p8 L8
p8 L12
p9 L4
p9 L8
p9 L12
p10 L4
p10 L8
p10 L12
p11 L4
p11 L8
p11 L12
p12 L4
p12 L8
p12 L12
p13 L4
p13 L8
p13 L12
p14 L4
p14 L8
p14 L12
p15 L4
p15 L8
p15 L12
token position (generated span)
top-1 expert id
0
10
20
30
40
50
60


Top-1 expert over generated tokens, 16 prompts x layers [4, 8, 12]
(model=OLMoE-1B-7B-0125-Instruct, variant=sft_baseline)
p0 L4
p0 L8
p0 L12
p1 L4
p1 L8
p1 L12
p2 L4
p2 L8
p2 L12
p3 L4
p3 L8
p3 L12
p4 L4
p4 L8
p4 L12
p5 L4
p5 L8
p5 L12
p6 L4
p6 L8
p6 L12
p7 L4
p7 L8
p7 L12
p8 L4
p8 L8
p8 L12
p9 L4
p9 L8
p9 L12
p10 L4
p10 L8
p10 L12
p11 L4
p11 L8
p11 L12
p12 L4
p12 L8
p12 L12
p13 L4
p13 L8
p13 L12
p14 L4
p14 L8
p14 L12
p15 L4
p15 L8
p15 L12
token position (generated span)
top-1 expert id
0
10
20
30
40
50
60

Figure 9: OLMoE-1B-7B, working-set reward alone — expert routing trace at layers 4/8/12.

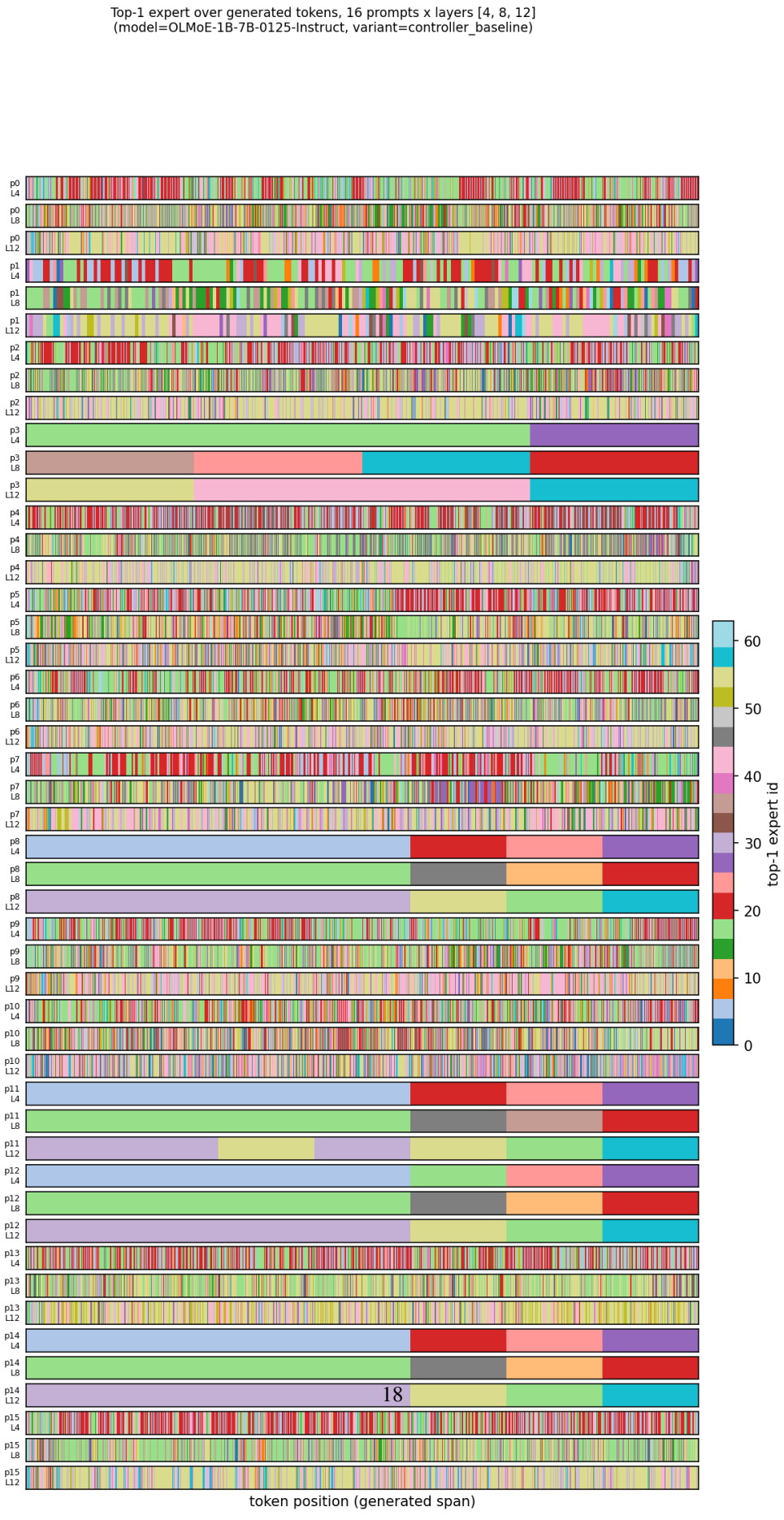

Figure 10: OLMoE-1B-7B-option_critic-controller_baseline — expert routing trace at layers 4/8/12


Top-1 expert over generated tokens, 16 prompts x layers [4, 8, 12]
(model=OLMoE-1B-7B-0125-Instruct, variant=melinoe_baseline)
p0 L4
p0 L8
p0 L12
p1 L4
p1 L8
p1 L12
p2 L4
p2 L8
p2 L12
p3 L4
p3 L8
p3 L12
p4 L4
p4 L8
p4 L12
p5 L4
p5 L8
p5 L12
p6 L4
p6 L8
p6 L12
p7 L4
p7 L8
p7 L12
p8 L4
p8 L8
p8 L12
p9 L4
p9 L8
p9 L12
p10 L4
p10 L8
p10 L12
p11 L4
p11 L8
p11 L12
p12 L4
p12 L8
p12 L12
p13 L4
p13 L8
p13 L12
p14 L4
p14 L8
p14 L12
p15 L4
p15 L8
p15 L12
token position (generated span)
top-1 expert id
0
10
20
30
40
50
60

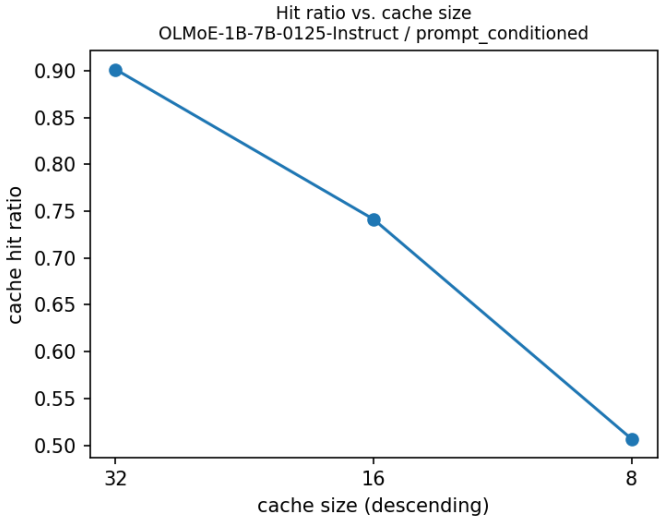

Figure 12: OLMoE-1B-7B, prompt-conditioned policy – working-set hit ratio as a function of the conditioned capacity $c \in \{8, 16, 32\}$ (descending on the x-axis), corresponding to Table 3.

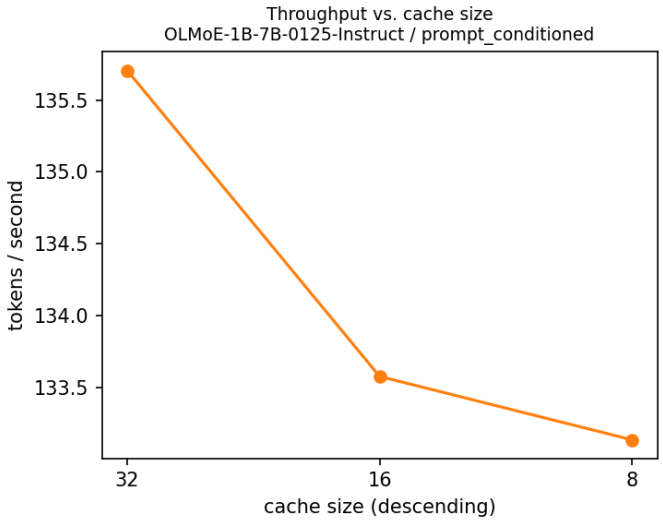

Figure 13: OLMoE-1B-7B, prompt-conditioned policy – estimated tokens/second (own, adapter-inflated measurement, not base-normalized) as a function of the conditioned capacity $c \in \{8, 16, 32\}$ (descending on the x-axis), corresponding to Table 3.